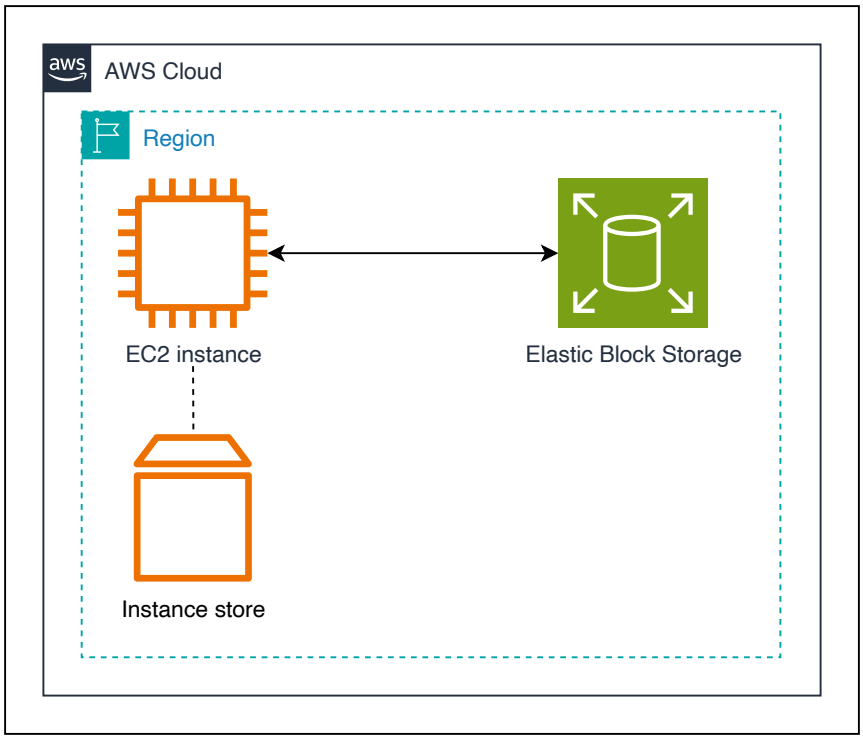
EC2 Instance Store Overview

Explore how EC2 instance store provides ephemeral, high-performance storage directly attached to the host machine. Understand its characteristics, use cases like caching and batch processing, and how it differs from persistent Amazon EBS. Learn to apply instance store in stateless, resilient AWS architectures effectively while managing its data loss risks.

When an EC2 instance launches, depending on the instance type, AWS provides access to what's known as instance store volumes. These are physical disks directly attached to the host machine that runs on a virtual instance. Unlike Amazon EBS (Elastic Block Store), a network-attached storage option designed for persistence, instance store is ephemeral.



EC2 association with EBS and instance store

Ask

Instance store volumes are ephemeral and deliver extremely high performance, especially IOPS, because the data does not have to traverse a network. This makes them valuable in situations where temporary, high-throughput storage is beneficial.

In the next section, we'll take a closer look at how instance store differs from Amazon EBS in terms of functionality and usage.

How instance store differs from Amazon EBS

Although both Amazon EBS and instance store provide block-level storage for EC2 instances, their behavior, lifecycle, and performance profiles are significantly different.

Amazon EBS is designed for persistent storage. This means our data is retained even after the EC2 instance stops or is terminated, making it ideal for databases, long-term logs, and file systems where durability is a requirement. We can detach and reattach an EBS volume to different instances, create snapshots, and scale the storage independently of the instance's lifecycle.

Instance store vs. EBS

Instance store, by contrast, is ephemeral. The storage is available only while the EC2 instance is running on the same underlying physical host. If the instance is stopped, terminated, or migrated, the data is irreversibly lost. This makes it unsuitable for persistent storage. However, an instance store can achieve extremely low latency and high IOPS for temporary data processing.

Ask

- **Performance comparison:** Because instance store is physically local to the host, there is no network latency, making it ideal for workloads that require ultra-fast disk access. Think of temporary scratch space during batch processing or data transformations.
- **Cost consideration:** EBS volumes are billed separately, based on their size and throughput configuration. Instance store, on the other hand, is included in the cost of supported EC2 instance types. This can be advantageous in scenarios where high-performance, short-term storage is needed and cost efficiency is critical.
- **Flexibility and management:** EBS offers snapshots, encryption at rest with AWS KMS, and the ability to resize volumes. Instance store offers none of these out-of-the-box. The developer must handle encryption and data lifecycle manually.

Feature	Amazon EBS	EC2 Instance Store
Persistence	Persistent across stops and reboots	Lost on stop, terminate, or host failure
Performance	Network-attached, variable latency	Local disk, high IOPS, and low latency
Cost	Billed separately per volume	Included in instance cost
Attach/detach	Can attach/detach from instances	Tied to the lifecycle of one instance
Snapshots and backups	Supports snapshots and backups	No built-in backup mechanism
Encryption	Integrated with KMS	Must be implemented manually
Suitable for	Databases, logs, and persistent storage	Caches, scratch data, and temporary files

Ask

Next, we'll look at the defining characteristics and limitations of instance store that every developer should consider.

Characteristics and limitations

Instance store volumes are not universally available across all EC2 instance types. Only certain families, such as `i3`, `d3`, `h1`, and selected bare-metal instances support them. This means that before choosing instance store for an application, we must validate that our instance type provides such volumes.

One of the most important limitations is that instance store cannot be used to store any data that needs to persist. If the host machine fails or an instance is stopped or terminated, all the data is lost. Even a simple reboot that relocates the instance to a different physical host will result in data loss. As a result, developers must use these volumes only for temporary storage and ensure that critical data is replicated or backed up elsewhere.

Also, unlike EBS volumes, instance store volumes cannot be resized or moved. Their capacity is fixed at the time of instance launch and cannot be changed.

Encryption is not provided by default, and AWS does not support automatic snapshotting of instance store data. If encryption is required, we need to set up our own disk-level encryption using operating system tools like `dm-crypt` or third-party solutions.

In the next section, we'll explore which workloads can best leverage the high-throughput, low-latency nature of instance store.

Use cases for instance store

Instance store is best suited for scenarios where high-performance, non-persistent storage is acceptable or even advantageous. Because the data doesn't persist beyond the instance lifecycle, it's ideal for workloads that either regenerate their data or rely on external data stores for persistence.

Ask

- **High-speed caching:** Instance store is ideal for applications like web accelerators, reverse proxies, or distributed caches using tools such as

Varnish or Redis (non-persistent mode) where rapid data access is crucial.

- **Temporary scratch space:** It works well for data processing tasks like big data jobs, media encoding, rendering, or scientific modeling, where intermediate files can be safely discarded after processing.
- **ETL (extract, transform, load) jobs:** ETL jobs benefit by using instance store for staging extracted data that doesn't require durable storage, which helps in reducing I/O latency and cost.
- **Machine learning workloads:** These can use instance store for preprocessing large datasets, as intermediate results can be lost without impacting the final model saved to persistent storage.
- **Build servers:** Instance store can be used for temporary directories or compilation artifacts, reducing runtime and costs.
- **Swap space:** We can also use instance store for swap space in memory-constrained environments. However, it's critical to monitor memory pressure and avoid over-reliance on swap, as it can degrade performance.

Hybrid configuration with EBS

A common design pattern is to combine instance store with EBS in a hybrid configuration: use EBS for persistent application data, and instance store for temporary operational files such as logs, buffers, or swap space. Developers often mount instance store volumes to standard Linux directories like `/mnt` or `/media/ephemeral0`, though we can choose our mount point and formatting preferences during initialization. It's common to automate this setup using EC2 user data scripts, which execute at boot time to format and mount the disk and perhaps copy in any required starter files.

For example, a video encoding pipeline may write raw ingest files to instance store while keeping the finished outputs on Amazon S3. Always ensure tha configuration files, or other runtime data essential for system recovery are written to persistent storage. This hybrid patter—ephemeral where it's safe, durable

Ask

where it’s critical—is a hallmark of resilient AWS application design. The illustration below shows how we can use instance store as a temporary storage for temporary operational files:

Using instance store for temporary operational files

Next, we'll explore how we can integrate the instance store to our application designs.

Stateless and cache-based workload patterns

One of the best matches for instance store is a stateless workload design, where each EC2 instance operates independently and doesn’t rely on persistent state. When an architecture supports replacing or removing instances without service disruption, it becomes easier to take advantage of instance store’s ephemeral nature.

Ask

Stateless applications, such as web servers behind a load balancer or workers in a batch-processing fleet, can freely use instance store for logs, temporary output, or intermediate data. Because there is no reliance on the instance's storage for long-term data retention, these workloads can scale horizontally and tolerate unexpected interruptions or terminations.

Cache-based systems also align perfectly with instance store. Caches are naturally ephemeral by design. If a cache node disappears, the system simply repopulates it from the source of truth. Tools like Memcached or Redis (in non-persistent configuration) can be deployed on EC2 with instance store backing, providing low-latency performance and reducing the cost of storage for transient data.

We can also design batch processing systems where instances read raw data from S3, perform transformations using temporary files on instance store, and then write the results back to S3 or a database. This lets us minimize EBS costs while maintaining strong I/O performance throughout the workflow. The illustration below shows how we can manage the data access using instance store:

Ask

Storing frequently accessed data to instance store

Conclusion

Effectively using EC2 instance store means understanding its ephemeral nature: data is lost if an instance stops or fails, so always replicate critical information to persistent storage like S3 or EBS. It lacks built-in encryption, requiring manual OS-level solutions for sensitive data. While AWS clears data, secure erasure for compliance needs OS-level tools. Ultimately, instance store is best for temporary data in resilient, stateless architectures (like Auto Scaling groups), where automated monitoring and recovery ensure continuity.

Ask